

LEC21 - Efficiency Considerations

__contains__ in a List

Suppose v refers to a number. How efficient is the following statement in its use of time?

```
v in [97, 36, 48, 73, 156, 947, 56, 236]
```

- Worst case is $O(n)$, best case is $O(1)$
 - If the list was 10x longer, the search would take 10x longer in the worst case
 - With either a Python list or a linked list of n elements...
 - We must look at elements one by one
 - Each time we eliminate only one element from consideration
 - In the worst case, we look at all n elements
 - Regardless, it takes time proportional to n
-

Making __contains__ Faster

- Idea is to add order (making the list sorted in ascending order in this case)
 - This allows us to use Binary Search, which reduces the worst case time to $O(\log_2 n)$
 - $O(\log_2 n)$ is also known as $O(\log n)$ in computer science
 - Check the middle element of the list, if it is greater, check repeat with the right side. Otherwise, repeat with the left side
-

Multiset ADT (search, insert, delete)

- For a sorted list with n items...
 - Searching is fast $\rightarrow O(\log n)$ because of Binary Search
 - Insertion and deletion can be slow, if inserting/removing from the front of the list $\rightarrow O(n)$
-

Tree Searching

How efficient is `__contains__` on each of the following...

1. General Tree class

```
for subtree in self._subtrees:
    if item in subtree: # if subtree.__contains__(item)
        return True
return False
```

LEC21 - Efficiency Considerations

- We need to look through all elements one by one
- Worst case looks at all elements
- Therefore, the worst case is $O(n)$

In general...

- Insertion can be fast, if you insert as a child of the root $\rightarrow O(1)$
- Searching and deletion can be very slow, since you might need to check every item in the tree $\rightarrow O(n)$

2. BinarySearchTree class

- If the binary search tree is “balanced” (has relatively equal number of nodes in left and right subtrees), then searching, insertion, and deletion can run most efficiently on average since half of the tree can be eliminated with each pass $\rightarrow O(\log n)$
- If the tree is not balanced (has more nodes on one side), then all operations need to check all nodes $\rightarrow O(n)$

Operation	Sorted List	Tree	Binary Search Tree
Search	$O(\log n)$	$O(n)$	$O(\log n)$
Insertion	$O(n)$	$O(1)$	$O(\log n)$
Deletion	$O(n)$	$O(n)$	$O(\log n)$